RELAZIONE CENTRALINA AERAZIONE DOMESTICA

Ferrari Kevin, Matricola: 130668

Corso di Architettura degli Elaboratori - Modulo Laboratorio

Indice

ANALISI.....3
1. 1 Testo assegnato.....3
1.2 Analisi del problema.....4
1.2.1 Obiettivi principali:.....5
1.2.2 Analisi Scenari.....5
1.2.3 Componenti del sistema.....7
1.2.4 Ambiguità del sistema.....7
1.2.5 Decisione dei scenari.....8
1.2.6 Risveglio del sistema.....10
1.2.7 Considerazioni sul sistema.....10
1.2.8 Diagrammi di flusso.....11
1.3 Analisi della soluzione adottata.....13
1.3.1 Descrizione del Diagramma degli stati e comportamenti.....14
1.3.2 Gestione delle interruzioni.....15
1.4 Analisi delle soluzioni alternative.....16
SINTESI.....18
2.1 MOD1: Acquisizione livello CO2.....18
2.2 MOD2: SENSORI DHT22.....19
2.3 MOD3: Attivazione recupero calore.....20
2.4 MOD4: Gestione Console di Comando (Impostazione parametri).....21
SISTEMA COMPLESSIVO.....23
3.1 Panoramica del codice.....24
3.1.1 Definizione delle variabili globali.....24
3.1.2 Assegnazione dei PIN e Dispositivi.....25
3.1.3 Funzionamento del void setup().....26
3.1.4 Funzionamento del void loop().....26
3.2 Considerazioni finali.....26

CAPITOLO 1

ANALISI

1.1 Testo assegnato

Realizzare un sistema di ricambio aria per abitazione in grado di:
a) Attivarsi quanto il livello di CO2 supera una soglia predeterminata
b) Scegliere la modalità di ventilazione più economicamente vantaggiosa tra:
b1) a recupero di calore interno (fresco o caldo)
b2) scambio diretto del calore con l'esterno (fresco o caldo)
Per questo, si dovranno usare due rilevatori della temperatura, tramite https://docs.wokwi.com/parts/wokwi-dht22 uno da porre all'esterno ed uno all'interno.
e 2 Relay: https://docs.wokwi.com/parts/wokwi-relay-module connessi, rispettivamente a:
a) ventola di aspirazione aria esterna ed espulsione aria interna
b) deviatore flusso di ventilazione su circuito a recupero di calore
Ai fini del progetto, simuliamo il sensore CO2 con un potenziometro
Il sistema dovrà una console di comando costituita da:
a) Display da minimo 16 colonne x 2 righe
b) 4 tasti (< > S X) per impostare i parametri e le opzioni

La progettazione dovrà essere svolta secondo il paradigma:
•Minori risorse impiegate per la macchina
•Maggior sforzo per la progettazione e sviluppo.
In particolare:
a) I componenti HW-SW dovranno essere "Automi" indipendenti che comunicano con variabili globali
b) Le cadenze temporali (ritardi ecc.) dovranno essere governati dall'ISR del clock
c) La progettazione delle ISR dovrà garantire tempi di servizio compatibili con le risposte al tempo reale del sistema.
d) La gestione degli I/O deve essere diretta, senza usare le funzioni di "Arduino" o librerie esterne.

1.2 Analisi del problema

Il progetto in esame consiste nello sviluppo di una centralina di aerazione domestica capace di controllare la qualità dell'aria all'interno di un appartamento tramite una rete di sensori dedicati.
Il sistema deve essere in grado di eseguire le seguenti operazioni in sequenza:

A. **ACQUISIZIONE DEL LIVELLO DI CO2**
   Il sistema analizza costantemente un parametro molto sensibile, ovvero il **livello di CO2**, misurato in parti per milione che sarà acquisito da un sensore.
   Questi livelli variano in base all'ambiente e alla funzione dello spazio. Dato che questo progetto si focalizza su ambienti chiusi, ho individuato dei livelli di CO2 da poter affrontare e sviluppare nel corso del progetto:
   - **400-450 ppm**: è il livello naturale nell'atmosfera esterna.
   - **600-800 ppm**: è ideale per ambienti interni ben ventilati, come abitazioni o uffici. Questo livello è considerato confortevole per gli occupanti.
   - **800-1000 ppm**: è accettabile in ambienti affollati ma in questo contesto del progetto potrebbe causare una sensazione di aria viziata.
   - **1000-1500 ppm**: indica una ventilazione insufficiente.
   - **>1500 ppm**: indica un livello elevato di CO2 e converrebbe un intervento immediato o segnalazione tramite allarme.
B. **CONTROLLO DEL LIVELLO DI CO2 CON LA SOGLIA MAX**
   Analizzando i livelli descritti nella prima operazione, ho considerato come **livello ottimale 1000 ppm** in cui se **il sistema supera** questo livello massimo, allora **si accende (VENTILAZIONE ON)**. Altrimenti se il **livello di CO2 non supera** il livello ottimale, il **sistema si spegne** entrando in una modalità sleep, ottimizzando il consumo di energia **(VENTILAZIONE OFF).**
   Nel caso eventuale in cui questo livello di CO2 non si abbassa mai, è importante implementare un allarme per segnalare la brutta qualità dell'aria oppure il malfunzionamento del sistema.
C. **ATTENDERE TEMPO DI ATTESA E RIPARTIRE DA A**
   E' importante rilevare costantemente il livello di CO2 per rendere il sistema dinamico e sicuro, perciò una volta trascorso un **tempo di attesa**

(esempio 5 secondi) il sistema dovrà rilevare il livello di CO2 e quindi ripartire dall'operazione A.

Per quanto riguarda il **sottosistema di VENTILAZIONE** che può essere ON oppure OFF, deve eseguire le seguenti operazioni in sequenza:

A. **SISTEMA DI VENTILAZIONE ON**
   Se il sistema di ventilazione è acceso, allora:
   a. Se recupero calore, che ha come valori di ingresso:
      - Temperatura interna
      - Temperatura esterna
      - Temperatura di benessere (intervallo di comfort)

      Mi ritorna **true**, allora il **Deviatore del recupero calore è ON**, altrimenti Deviatore di recupero calore è OFF.
   b. La ventola rimane ON, perchè il sistema è acceso.

B. **SISTEMA DI VENTILAZIONE OFF**
   La ventola rimane spenta in quanto il sistema è spento.

1.2.1 Obiettivi principali:

- **Massima efficienza energetica**: L'MCU deve entrare in sleep-mode quando possibile e risvegliarsi solo in caso di interrupt.
- **Gestione in tempo reale**: uso di interruzioni e dei timer per gestire ritardi
- **Risposta rapida ed efficace**: distinguere correttamente gli eventi delle modalità di ventilazione

1.2.2 Analisi Scenari

Possiamo effettuare degli esempi pratici che rappresentano a pieno le varie casistiche del sistema. Impostiamo una temperatura di benessere tra 20°C e 25°C:

1. **SCENARIO 1: INVERNO**
   Parametri:
   - **Temperatura Interna**: 21°C
   - **Temperatura Esterna**: 2°C
   - **Confronto**: la Temperatura esterna è FUORI dall'intervallo di benessere.
   - **Decisione**: recupero di calore torna true → Deviatore RC ON

In questo scenario l'aria viziata 21°C che esce cede il suo calore all'aria gelida a 2°C che entra. L'aria nuova non entrerà mai a 2°C ma magari a 16°C.

2. **SCENARIO 2: ESTATE**
   Parametri:
   - **Temperatura Interna**: 24°C
   - **Temperatura Esterna**: 34°C
   - **Confronto**: La temperatura esterna è FUORI dall'intervallo di benessere
   - **Decisione**: recupero di calore torna true → Deviatore RC ON
     In questo scenario l'aria fresca a 24°C che esce cede il suo "fresco" all'aria torrida a 34°C che entra. L'aria nuova non entrerà a 34°C ma magari a 27°C.

3. **SCENARIO 3: MEZZA STAGIONE**
   Parametri:
   - **Temperatura Interna**: 26°C.
   - **Temperatura Esterna**: 22°C.
   - **Confronto**: la temperatura esterna è DENTRO l'intervallo di benessere (20-25°C).
   - **Decisione**: recupero calore ritorna false → Deviatore RC OFF
     Il sistema bypassa il recuperatore e fa entrare direttamente l'aria esterna a 22°C.

4. **SCENARIO 4: SISTEMA SPENTO O ACCESO**
   Qui indipendentemente dalle temperature rilevate, il sistema E' SPENTO dato che il livello di CO2 non supera la soglia max, altrimenti E' ACCESO.

1.2.3 Componenti del sistema

I componenti descritti in seguito godono di una propria autonomia e possiamo quindi svilupparli distintamente individuando gli elementi fondamentali:

- **LCo2:** lo scopo è acquisire il livello di CO2 da un sensore analogico e renderlo disponibile su una variabile globale definita LCo2 che rappresenta a pieno la richiesta del progetto.
- **Modalità recupero di calore**: deve essere verificata quando conviene attivare il recupero di calore prendendo in input la temperatura interna, la temperatura esterna e la temperatura di benessere
- **Temperatura interna**: acquisisce la temperatura interna dal sensore DHT22 fornito dal progetto.
- **Temperatura esterna**: acquisisce la temperatura esterna dal sensore DHT22 fornito dal progetto.
- **Console di comando**: permette l'impostazione dei parametri(descritti successivamente).

1.2.4 Ambiguità del sistema

Nel testo del progetto sono presenti alcune aree di incertezza che possono influenzare significativamente la progettazione del sistema. Di seguito si riportano le principali ambiguità individuate, accompagnate da osservazioni progettuali per una corretta interpretazione.

- **Controllo periodico non citato**: nel testo non viene spiegato esplicitamente ogni quanto deve essere effettuato il controllo dei valori. (es. dopo ogni mezz'ora, ogni ora, oppure anche minuti o secondi).
- **Spegnimento del sistema ambiguo**: non era ben chiaro nel testo l'effettivo "riposo" del sistema in caso di controllo tra livello di CO2 e soglia MAX.
- **Scelta della modalità:** la scelta della modalità di ventilazione era semplicemente citata senza andare nel dettaglio su come azionarla, ad esempio con una differenza tra le due temperature oppure un range di comfort per la temperatura di benessere.
- **Soglie per il CO2 e temperature**: nel testo il sistema non spiega chiaramente una soglia MAX da impostare rendendo quindi ambigua l'attivazione del sistema.

1.2.5 Decisione dei scenari

DECISIONE SCENARIO 1

Livello di Co2	Soglia di CO2	Temp Interna rilevata	Temp Esterna rilevata	Temp di Benessere	Output controllo CO2	Output
1050	1000	21	2	20-25	SI	Recupero di calore
1100	1000	25	5	20-25	SI	Recupero di calore
1020	1000	30	10	20-25	SI	Recupero di calore

Tabella 1.1 Tabella di decisione degli scenari dello scenario 1

DECISIONE SCENARIO 2

Livello di Co2	Soglia di CO2	Temp Interna rilevata	Temp Esterna rilevata	Temp di Benessere	Output controllo CO2	Output
1050	1000	24	34	20-25	SI	Recupero di calore
1100	1000	26	30	20-25	SI	Recupero di calore
1020	1000	20	26	20-25	SI	Recupero di calore

Tabella 1.2 Tabella di decisione degli scenari dello scenario 2

DECISIONE SCENARIO 3

Livello di Co2	Soglia di CO2	Temp Interna rilevata	Temp Esterna rilevata	Temp di Benessere	Output controllo CO2	Output
1050	1000	21	21	20-25	SI	Scambio diretto
1100	1000	25	25	20-25	SI	Scambio diretto
1020	1000	30	10	20-25	SI	Recupero di calore
1200	1000	29	24	20-25	SI	Scambio diretto

Tabella 1.3 Tabella di decisione degli scenari dello scenario 3

DECISIONE SCENARIO 4

Livello di Co2	Soglia di CO2	Temp Interna rilevata	Temp Esterna rilevata	Temp di Benessere	Output controllo CO2	Output
800	1000	21	2	20-25	NO	Sistema spento
500	1000	25	5	20-25	NO	Sistema spento
1020	1000	30	10	20-25	SI	Recupero di calore
1100	1000	20	21	20-25	SI	Scambio diretto

Tabella 1.4 Tabella di decisione degli scenari dello scenario 4

1.2.6 Risveglio del sistema

Il sistema deve entrare in funzione solo quando risulta realmente indispensabile. Nel caso in cui non si rileva un livello di CO2 abbastanza grande da superare la soglia di 1000 ppm e quindi richiedere l'intervento della ventilazione, la centralina passa a una modalità di basso consumo (sleep-mode). Per garantire la massima efficienza, la MCU rimane in sleep mode finché non riceve un segnale. Solo al verificarsi di un interrupt esterno la centralina "si risveglia" andando ad esaminare la situazione per rilevare un'altra volta il livello di CO2. Solo in quel momento la situazione viene analizzata per capire se con i controlli periodici i nuovi livelli di CO2 superano la soglia. Dopo aver verificato che non è necessario agire, oppure dopo aver gestito un particolare evento, la centralina torna automaticamente nella modalità di basso consumo(sleep-mode).

1.2.7 Considerazioni sul sistema

Il sistema deve essere progettato in modo da renderlo dinamico e facilmente estendibile senza dover effettuare modifiche importanti al sistema di base, come ad esempio gestire un sensore di allarme quando il livello di CO2 è troppo alto oppure configurare un buzzer per riprodurre suoni di emergenza.

1.2.8 Diagrammi di flusso

Al fine di evitare errori durante la progettazione, sono passato alla realizzazione dei diagrammi di flusso, detti anche flowchart che rappresentano l'evoluzione del sistema e tutte le varie casistiche.

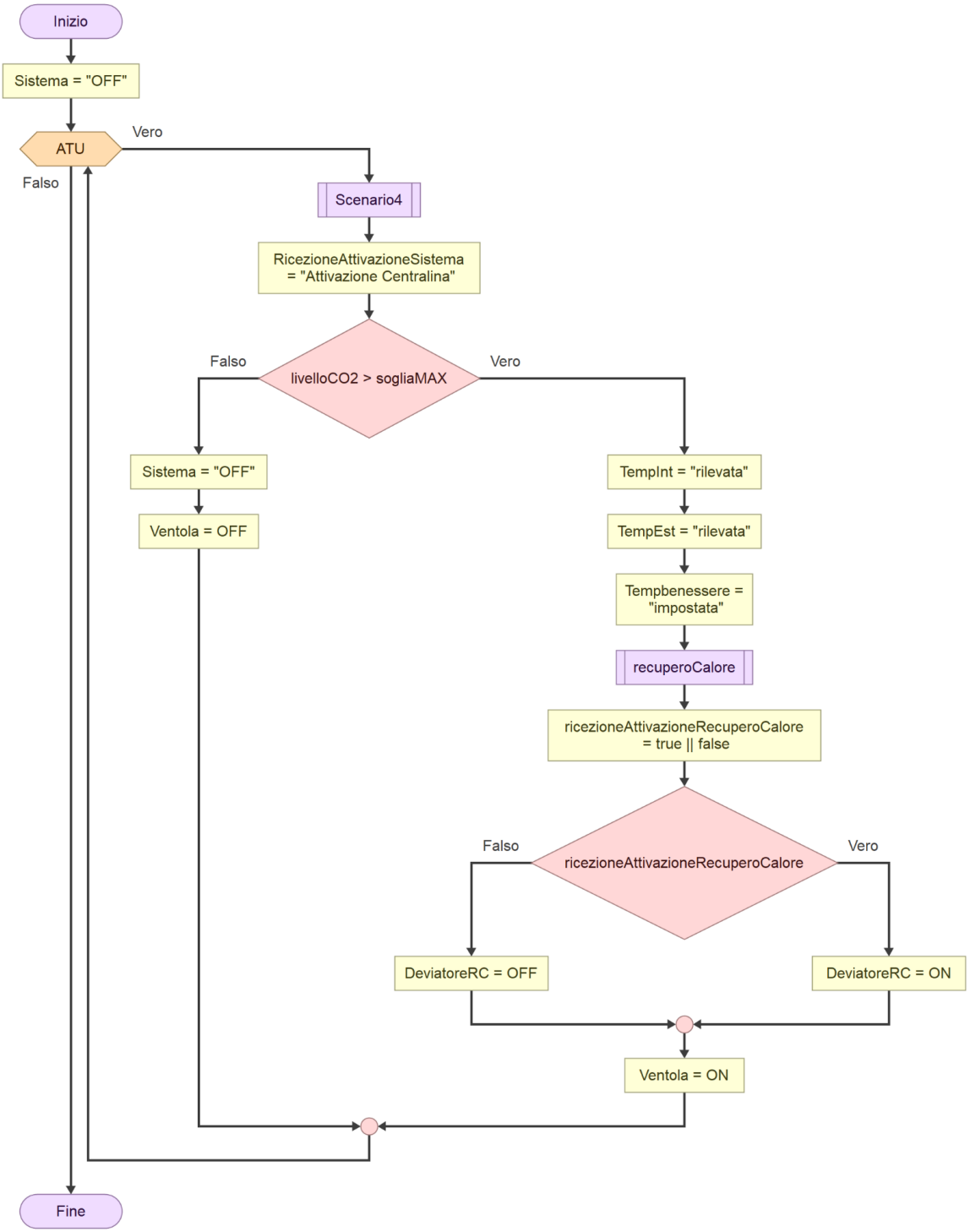

Figura 1.1 Diagramma di flusso che gestisce lo scenario della centralina

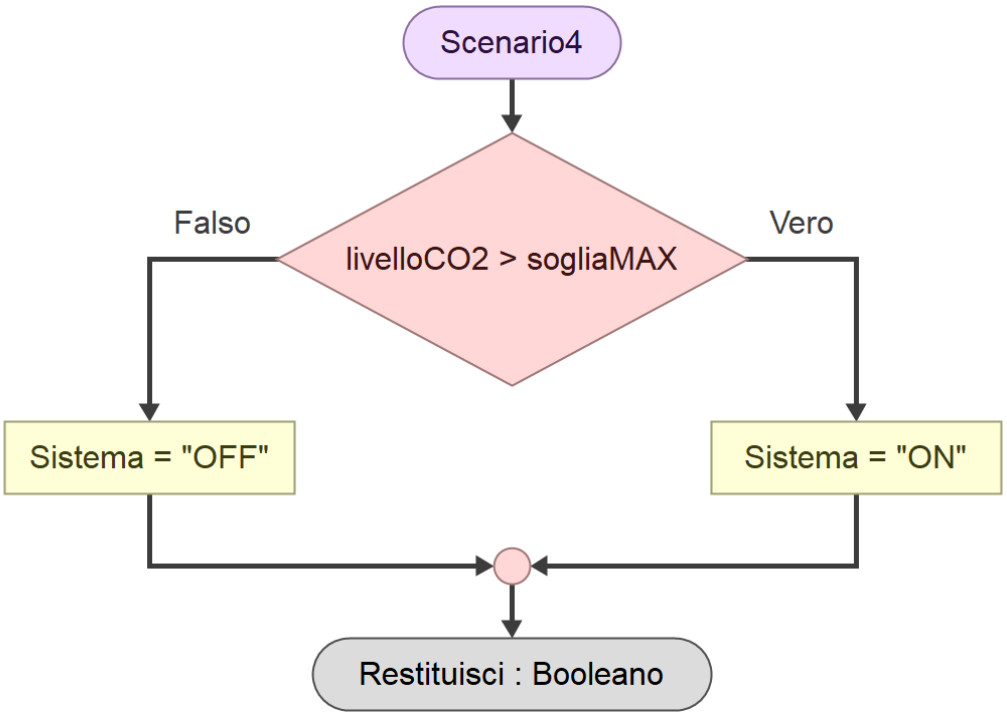

Figura 1.2 Diagramma di flusso che gestisce lo scenario 4

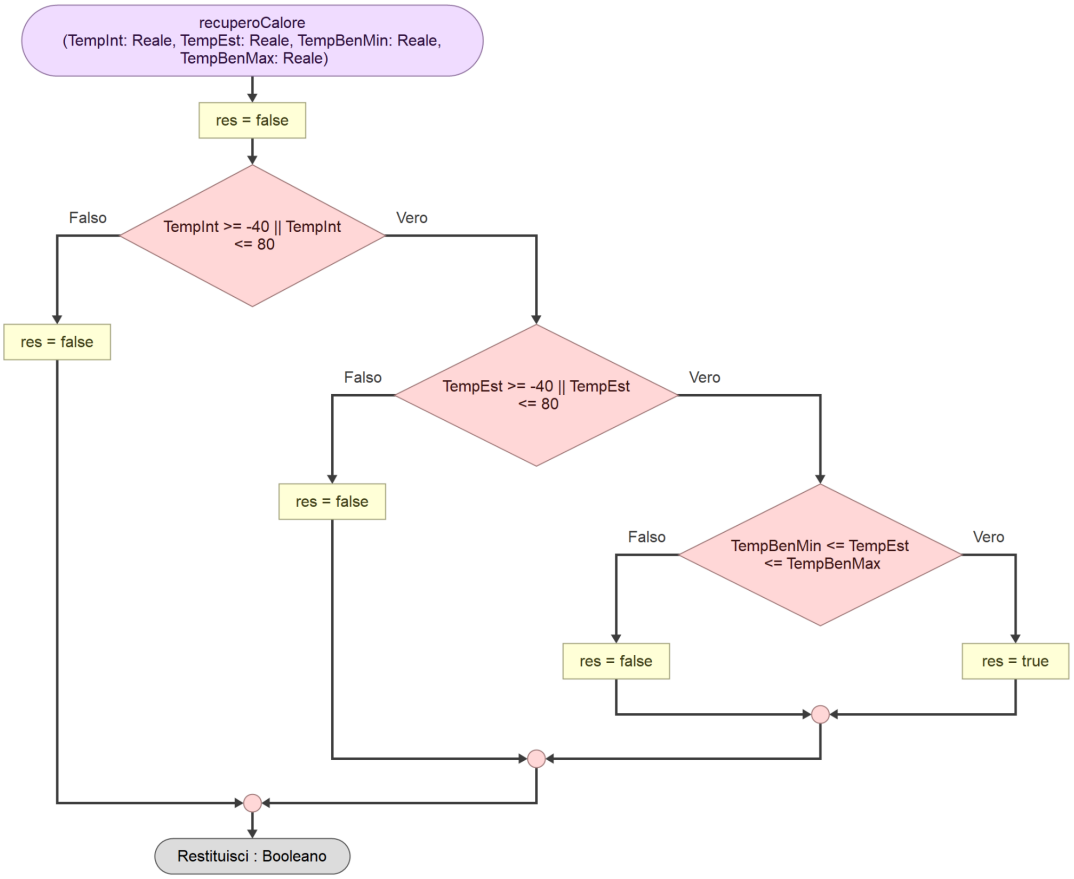

Figura 1.3 Diagramma di flusso che gestisce il recupero di calore

1.3 Analisi della soluzione adottata

Per descrivere al meglio in modo chiaro e non ambiguo il comportamento dinamico del sistema, esso è stato modellato come un automa a stati finiti, in cui ho individuato tutti gli stati in cui la centralina si trova, evidenziando soprattutto gli eventi che scaturiscono il cambiamento tra uno stato e l'altro, indicando anche con quali input e con quali output sono condizionati questi stati.
Si è scelto quindi di rappresentare graficamente questa logica attraverso un **diagramma degli stati** (statechart), che evidenzia i vari stati operativi del sistema. Esso aiuta a comprendere come il sistema evolve nel tempo al variare degli eventi.

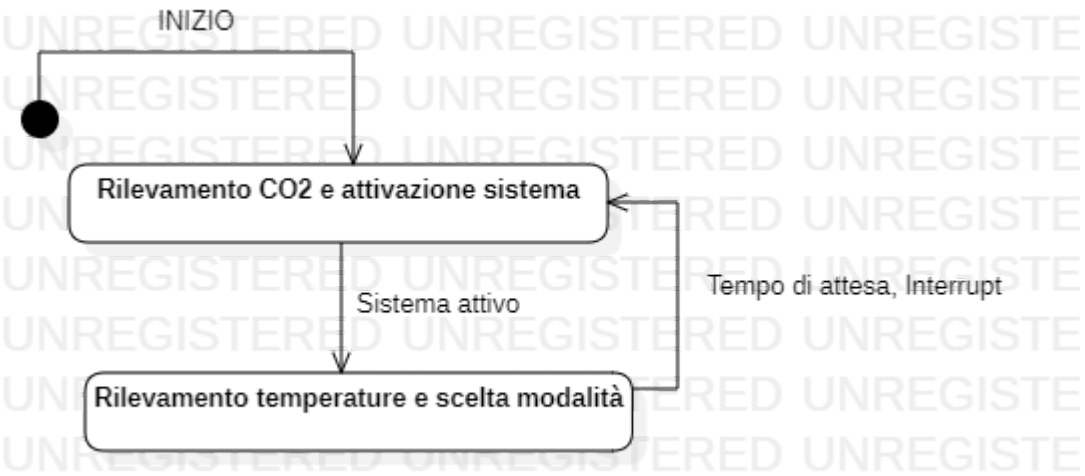

Figura 1.4 Diagramma degli stati del sistema complessivo

1.3.1 Descrizione del Diagramma degli stati e comportamenti

Come possiamo notare, il seguente diagramma degli stati descrive in maniera precisa e semplice il funzionamento del sistema. A ciascuno è stata associata una descrizione dettagliata, ma soprattutto saranno specificati il **COME** questi stati variano e **CON QUALI VALORI** sono definiti.
Partiamo con la descrizione del diagramma:

- **NUMERO DI STATI RILEVATI CON INPUT E OUTPUT**: partiamo con l'individuazione del numero di stati. Essi sono **3** in quanto il progetto consiste solamente in uno **stato iniziale** che rappresenta **l'inizializzazione dei componenti e del display.**
  Il **primo stato** consiste nella **rilevazione del livello di CO2 e confrontarlo con la soglia MAX.** Esso ha come input il livello di CO2 e la soglia impostata dall'utente e come output restituirà TRUE oppure FALSE che andrà a decidere l'attivazione del sistema.
  Il **secondo stato**, in quanto sistema attivo, consiste nella **rilevazione delle temperature** in cui base alla temperatura di benessere verrà attivata se conviene il recupero di calore. Questo stato ha come input le due temperature rilevate dai sensori e la temperatura di benessere e come output mi dirà TRUE se conviene attivare il recupero di calore, FALSE altrimenti.

- **VARIAZIONE DA UNO STATO ALL'ALTRO**: in questo sistema bisognerà aspettare un **tempo di attesa**, in cui lo stato entra in sleep-mode, che una volta trascorso ritornerà come descritto nello statechart al primo stato. Il sistema infatti viene risvegliato da un interruzione per minimizzare il consumo e il numero di risorse usate per effettuare un secondo controllo della situazione.
  La **variazione invece tra il primo stato e il secondo** è determinato come spiegato precedentemente dal risultato del **controllo tra il livello di CO2 e la soglia MAX**

1.3.2 Gestione delle interruzioni

Per ottimizzare il monitoraggio dell'ambiente circostante e mantenere il sistema a basso consumo energetico, si è deciso di utilizzare la gestione degli interrupt. Poiché il microcontrollore funziona principalmente in modalità a risparmio energetico, è essenziale adottare una strategia basata su eventi esterni, consentendo al sistema di attivarsi solo quando strettamente richiesto. Sono stati configurati i seguenti tipi di interruzioni:

- **INTERRUPT DI TIMER PERIODICO**: Serve a scandire il tempo per i controlli periodici del sistema. È questo interrupt che permette al sistema di rileggere i valori dai sensori ogni 5 secondi, come descritto precedentemente

- **INTERRUPT CAMBIO DI STATO (PIN CHANGE INTERRUPT)**: È l'interrupt che rileva l'istante esatto in cui un utente preme uno dei quattro tasti della console di comando. Questo garantisce la massima reattività del sistema all'input dell'utente.

- **INTERRUPT DI TIMER PER IL DEBOUNCE**: Lavora in coppia con l'interrupt di cambio di stato per gestire il "rimbalzo" dei pulsanti. Dopo che il Pin Change Interrupt rileva una pressione, questo secondo interrupt misura il tempo necessario per il debounce, assicurando che una singola pressione venga contata una sola volta.

1.4 Analisi delle soluzioni alternative

- **Utilizzo di librerie di alto livello arduino**:

    - **Vantaggio**: Questa implementazione si avvale di librerie e funzionalità di alto livello fornite dall'ambiente di sviluppo Arduino, mirate a semplificare la stesura del codice, migliorare la comprensione e renderlo più facile da gestire. Le operazioni complesse, come l'accesso diretto ai registri o la configurazione delle interruzioni, vengono nascoste da queste librerie, consentendo anche a sviluppatori con una conoscenza limitata dell'hardware sottostante di lavorare efficacemente sul progetto.
    - **Svantaggio**: Sebbene le librerie di Arduino introducono strumenti avanzati che agevolano lo sviluppo, non si sono dimostrate completamente adatte a soddisfare le esigenze specifiche del progetto. L'obiettivo chiave del sistema è assicurare prestazioni elevate, in particolare per quanto riguarda la rapidità di risposta. Tuttavia, l'uso di queste librerie comporta un notevole consumo di risorse computazionali, sia per la CPU che per la memoria, riducendo l'efficienza globale. Questo può causare ritardi significativi, compromettendo la capacità del sistema di reagire tempestivamente in contesti critici. Inoltre, le librerie ad alto livello limitano le possibilità di ottimizzazione a basso livello e impediscono una configurazione dettagliata del comportamento del sistema, rendendo impossibile l'accesso diretto ai registri del microcontrollore, un aspetto essenziale per una gestione precisa dell'hardware.

- **Approccio alla logica cablata**: Significherebbe eliminare il microcontrollore e costruire l'intero sistema utilizzando componenti elettronici digitali discreti, dove il "programma" non è una sequenza di istruzioni software, ma è fisicamente "scolpito" nei collegamenti tra i componenti. Questo comporterebbe all'utilizzo di porte logiche (AND, OR, NOT, XOR) e dei flip-flop.
  Svantaggi:
    - **Complessità**: adottando questa soluzione il circuito diventerebbe molto difficile da gestire.

- **Mancanza di flessibilità**: in quanto una logica cablata, se ci dovesse essere anche la più minima variazione del progetto, richiederebbe la completa riprogettazione del sistema aumentando la complessità generale.

L'utilizzo degli **interrupt** e l'**interazione diretta con i registri** del microcontrollore sono stati scelti per ottimizzare l'efficienza del sistema. Gli **interrupt** consentono al dispositivo di rispondere in modo immediato agli input provenienti dai sensori, garantendo **tempi di reazione rapidi** e adeguati ai requisiti di un sistema di sicurezza. Inoltre, questa metodologia permette al microcontrollore di operare in modalità di risparmio energetico durante i periodi di inattività, minimizzando il consumo di energia. La **gestione diretta dei registri** elimina le inefficienze introdotte dalle funzioni di alto livello, **migliorando le prestazioni complessive** e offrendo un controllo più dettagliato e accurato sull'hardware.

CAPITOLO 2

SINTESI

Dopo aver esaminato il problema e identificato una soluzione praticabile, si procede con lo sviluppo dei singoli moduli funzionali, che verranno successivamente uniti per formare il sistema completo.

2.1 MOD1: Acquisizione livello CO2

Il **modulo MOD1** è responsabile dell'acquisizione del livello di CO_2 proveniente dal sensore. Il microcontrollore, tramite il suo convertitore analogico-digitale, acquisisce questo livello e lo memorizza in una variabile globale. Il compito del modulo è poi confrontare questo valore con una soglia preimpostata e restituire un risultato booleano (vero o falso) che indica se il limite è stato superato. Sarà la logica di controllo principale del sistema a utilizzare questo risultato per decidere se attivare la ventilazione ("SISTEMA ATTIVATO") o mantenerla spenta.
Implementazione: questa acquisizione viene attivata grazie a un timer, nello specifico TIMER1, che come abbiamo deciso, dopo ogni 5 secondi comunicherà al sistema di attivarsi e acquisire il nuovo livello di CO2. Quando questo timer scade, l'interrupt di **TIMER1** imposta un flag nel loop() per avviare le operazioni di lettura.

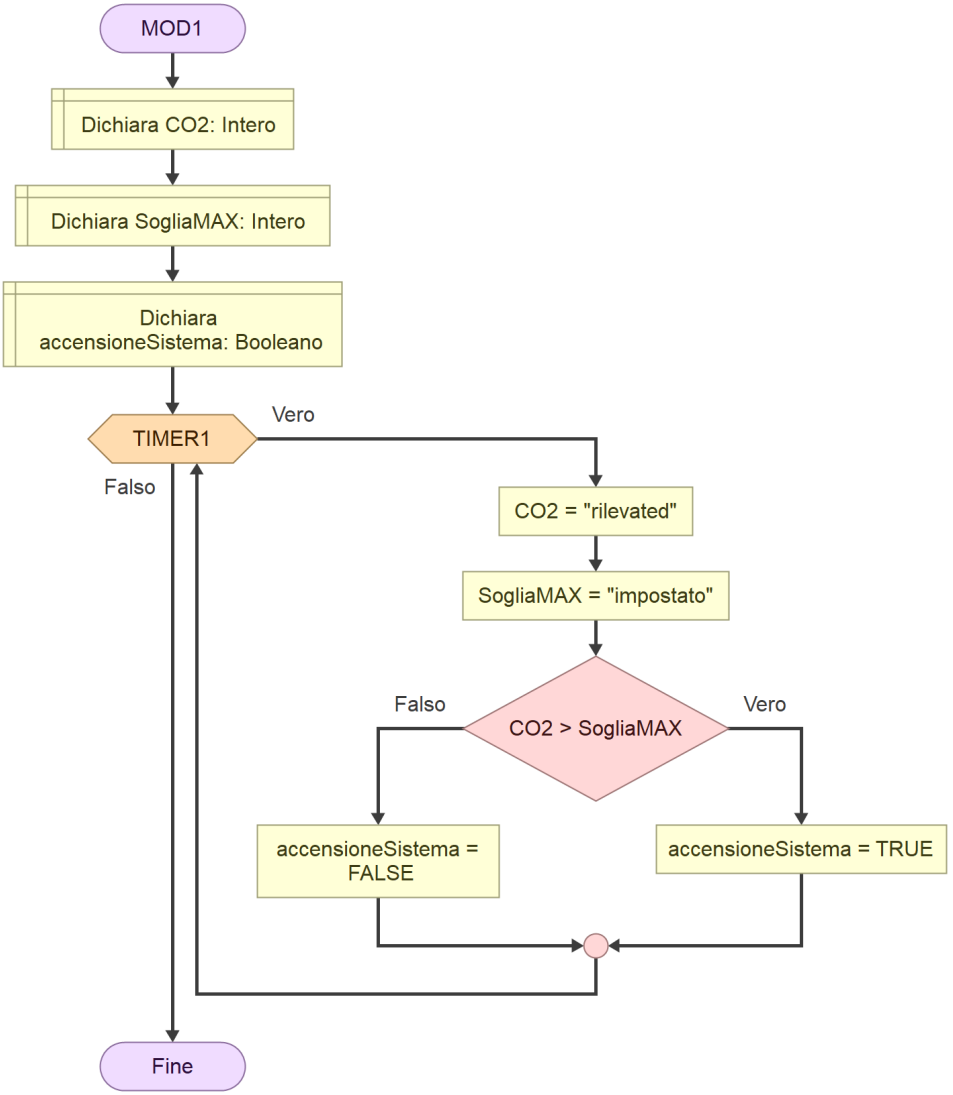

Figura 1.5 Flowchart del flusso logico del MOD1

2.2 MOD2: SENSORI DHT22

Il MOD2 gestisce i 2 sensori DHT22, impiegati come ingressi digitali per la rilevazione delle temperature interne e esterne.

Implementazione: questa implementazione è stata una delle parti più complesse perché non forniscono un valore di temperatura in modo semplice. Per ottenere la lettura, il microcontrollore deve avere una vera e propria "conversazione" con il sensore che invia i dati della temperatura un bit alla volta. Il valore di ogni bit (0 o 1) dipende dal tempo in cui il segnale rimane alto: un impulso lungo significa 1, uno corto significa 0. Ho utilizzato il TIMER0 del microcontrollore per misurare questi intervalli di tempo brevissimi.

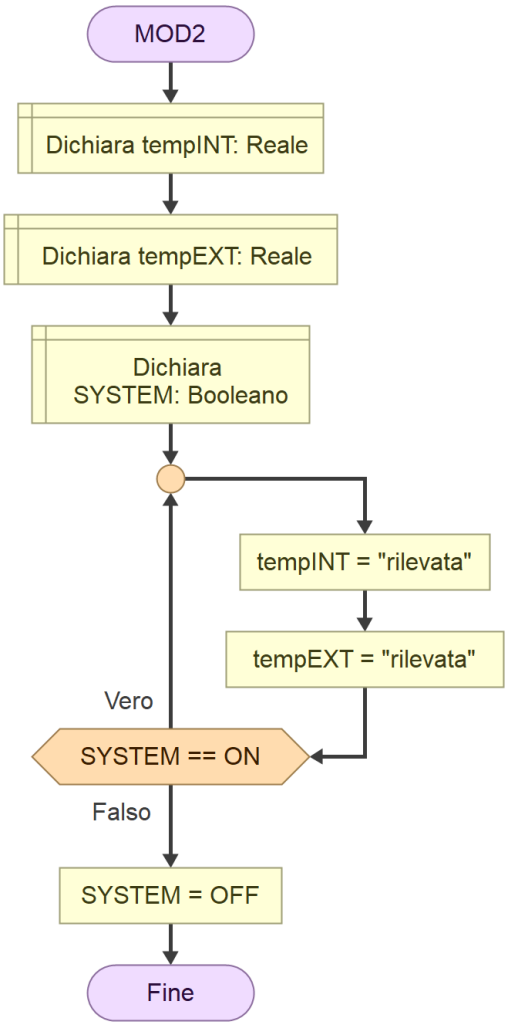

Figura 1.6 Flowchart del flusso logico del MOD2

2.3 MOD3: Attivazione recupero calore

Il **MOD3** ha il compito di gestire l'attivazione del deviatore del recupero. Ha come uscita un booleano ovvero true se conviene attivare il recupero di calore e in quel caso il deviatore RC viene acceso, altrimenti se è false il deviatore RC viene spento e messo ad OFF.
Implementazione: anche qui vengono sfruttati i timer hardware del microcontrollore come ad esempio è stato possibile simulare l'accensione del deviatore RC con il relè.

2.4 MOD4: Gestione Console di Comando (Impostazione parametri)

Il MOD4 ha il compito di gestire la console di Comando. Essa può avere diversi stati:

- **Stato MENU**: è lo stato iniziale della console in cui sarà possibile selezionare i parametri e eventualmente modificarli.
- **Stato MODIFICA SOGLIA MAX**: è lo stato in cui l'utente può modificare la soglia MAX del sistema per esigenze personali o anche tecniche.
- **Stato MENU TEMPERATURE**: è lo stato della console in cui è presente il range di comfort della temperatura di benessere. (TEMP MIN e TEMP MAX)
- **Stato MODIFICA TEMPERATURE**: è lo stato in cui l'utente può modificare il suo range di comfort delle temperature, sempre per esigenze personali

I parametri che possono essere configurati e perché sono stati riportati in questo modulo:

- **SOGLIA MAX**: è il parametro che rappresenta la soglia MAX(1000 ppm) che può essere configurata e salvata in una variabile durante l'avvio del sistema.
- **TEMPERATURA MINIMA DI BENESSERE**: rappresenta la temperatura minima che l'utente desidera nel range di comfort, modificabile sempre tramite la console.
- **TEMPERATURA MAX DI BENESSERE**: rappresenta la temperatura massima che l'utente desidera nel range di comfort, modificabile anche questa tramite la console.

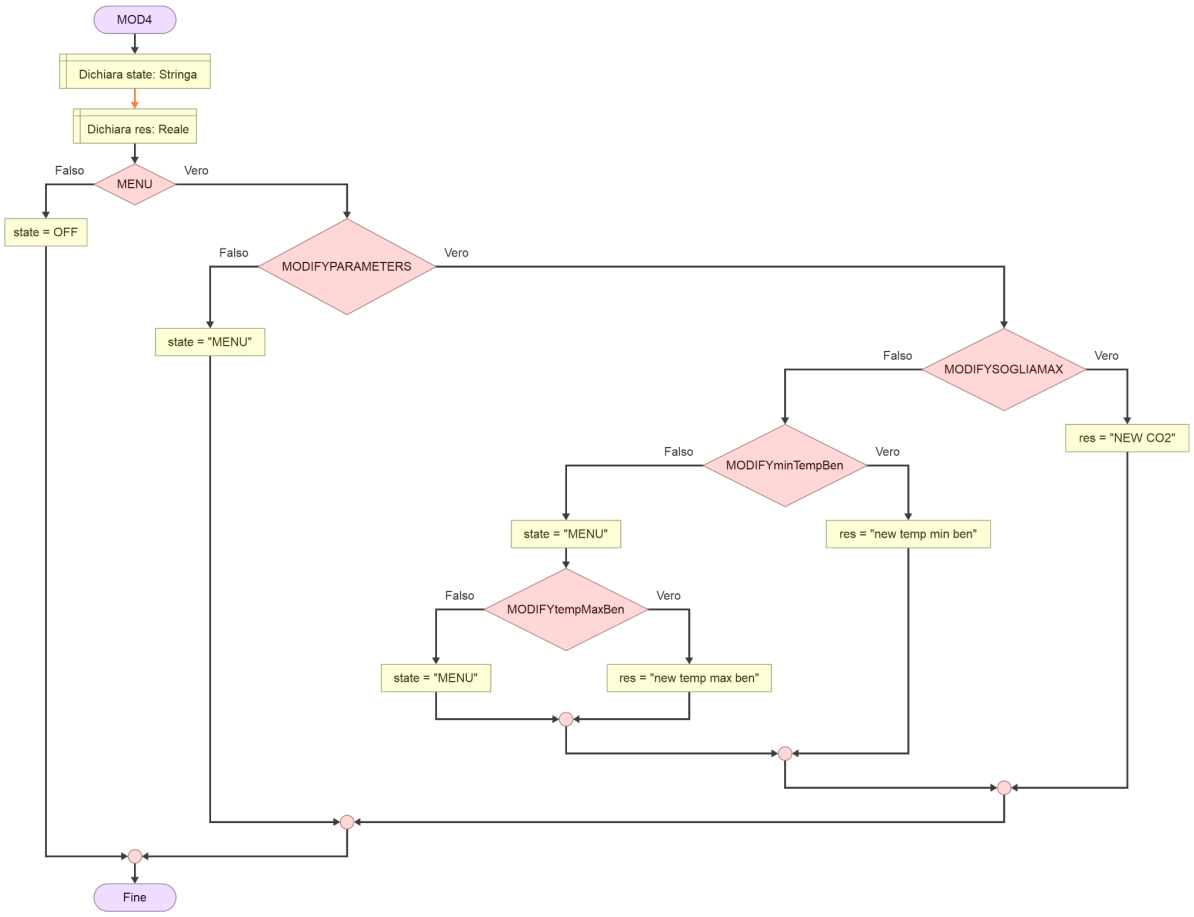

Figura 1.8 Flowchart del flusso logico del MOD4

CAPITOLO 3

SISTEMA COMPLESSIVO

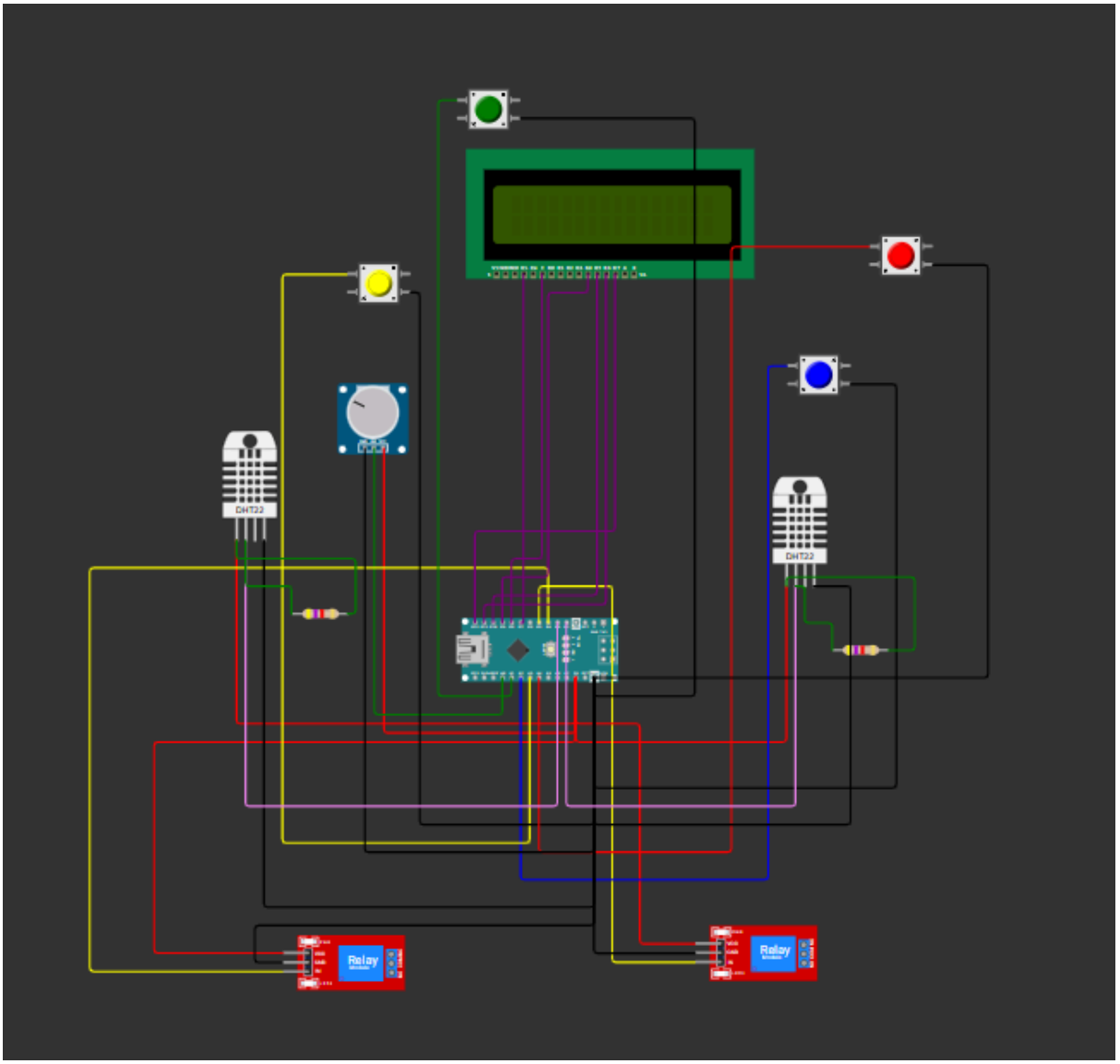

Figura 1.9 Sistema complessivo della centralina di aerazione domestica

A conclusione delle fasi di analisi e sintesi del progetto, il progetto completo con il codice della centralina è disponibile al seguente link:
https://wokwi.com/projects/434489889439706113

3.1 Panoramica del codice

3.1.1 Definizione delle variabili globali

All'interno del codice, come su richiesta del progetto, sono state definite le seguenti variabili globali usate all'interno del sistema per permettere ai componenti di comunicare tra di loro.

```C/C++
volatile uint8_t co2_level = 0; //Livello attuale di CO2
volatile uint8_t adc_ready = 1; //Flag di stato per il convertitore ADC
volatile uint8_t new_c02_data_available = 0; //Flag di comunicazione
dell'interrupt dell'ADC che lo imposta a 1 per segnalare che un nuovo valore
di CO2 è stato acquisito
volatile uint8_t update_display = 0; //È un flag per la gestione
dell'interfaccia utente. Viene impostato a 1 quando qualcosa cambia
volatile uint8_t read_co2_and_temp_flag = 0; //È il flag impostato
dall'interrupt periodico del Timer1. Viene impostato a 1 ogni 5 secondi per
comunicare al loop() che è il momento di eseguire la lettura periodica di
tutti i sensori
volatile uint8_t system_active = 0; //Rappresenta lo stato generale del
sistema di ventilazione. È 0 quando il sistema è spento e 1 quando è attivo
volatile int16_t temp_internal_x10; //Memorizza il valore della temperatura
del sensore di temperatura interno
volatile int16_t temp_external_x10;//Memorizza il valore della temperatura
del sensore di temperatura esterno
volatile uint8_t temp_read_status_internal = 1; //Flag di stato per indicare
il risultato dell'ultima lettura del sensore di temperatura
volatile uint8_t temp_read_status_external = 1; //Flag di stato per indicare
il risultato dell'ultima lettura del sensore di temperatura
volatile uint8_t menu_selection = 0; // Opzione selezionata nel menu (0 o 1)
volatile uint8_t co2_threshold = 1000;  // Soglia di CO2 per attivare il
sistema
volatile uint8_t temp_benessere_min = 20; // Valore di default per la T
minima di benessere
volatile uint8_t temp_benessere_max = 25; // Valore di default per la T
massima di benessere
```

3.1.2 Assegnazione dei PIN e Dispositivi

All'interno di questa sezione si stabiliscono le connessioni tra i pin della scheda e le componenti del sistema, specificando QUALI sono i PIN riservati ai sensori.

```C/C++
#define LCD_RS PD7 //Il pin "Register Select" del display è collegato al pin 7 della PORTA D.
#define LCD_E PB0 //Il pin "Enable" del display è collegato al pin 0 della PORTA B.
#define LCD_D4 PB1 //Il pin Dati 4 del display è collegato al pin 1 della PORTA B.
#define LCD_D5 PB2 //Il pin Dati 5 del display è collegato al pin 2 della PORTA B.
#define LCD_D6 PB3 //Il pin Dati 6 del display è collegato al pin 3 della PORTA B.
#define LCD_D7 PB4 //Il pin Dati 7 del display è collegato al pin 4 della PORTA B.
#define LCD_CLEARDISPLAY   0x01 // Comando per pulire lo schermo e riportare il cursore all'inizio.
#define LCD_FUNCTIONSET    0x20 // Comando base per impostare la modalità di funzionamento.
#define LCD_DISPLAYCONTROL 0x08 // Comando base per controllare l'accensione del display, cursore, ecc.
#define LCD_ENTRYMODESET   0x04 // Comando base per definire come il cursore si sposta dopo la scrittura.
#define LCD_SETDDRAMADDR   0x80 // Comando base per impostare la posizione del cursore.
#define LCD_2LINE          0x08 // Opzione per usare 2 linee del display.
#define LCD_DISPLAYON      0x04 // Opzione per accendere il display.
#define LCD_CURSOROFF      0x00 // Opzione per nascondere il cursore.
#define LCD_BLINKOFF       0x00 // Opzione per disattivare il lampeggio del cursore.
#define LCD_4BITMODE       0x00 // Opzione per la modalità di comunicazione a 4 bit.
#define LCD_5x8DOTS        0x00 // Opzione per usare caratteri di dimensioni 5x8 pixel.
#define DHT1_INTERNAL_PIN PD3 // Il sensore di temperatura interno è collegato al pin 3 della PORTA D.
#define DHT2_EXTERNAL_PIN PD2 //Il sensore di temperatura esterno è collegato al pin 2 della PORTA D.
#define RELAY_VENTILATION_PIN_NUM PD4 //Il relè per la ventilazione è collegato al pin 4 della PORTA D.
#define RELAY_HEAT_RECOVERY_PIN_NUM PD5 //Il relè per il recupero di calore è collegato al pin 5 della PORTA D.
```

3.1.3 Funzionamento del void setup()

La funzione **void setup()** è il punto d'ingresso di inizializzazione del sistema. Si occupa di gestire la configurazione dei componenti necessari per garantire il funzionamento della centralina di aerazione. In particolare:

- Inizializza una console seriale per vedere l'attivazione del sistema e l'attivazione del recupero di calore
- Inizializza tutti i timer
- Inizializza le variabili di stato
- Attiva gli interrupt globali

3.1.4 Funzionamento del void loop()

La funzione **void loop()** a differenza di setup gestisce il comportamento continuo del sistema e in particolare:

- Gestisci gli eventi di lettura dei valori grazie a un "flag" che viene aggiornato dall'interrupt per mandare il segnale di ricezione valori.
- Si gestisce il display
- Si gestisce lo stato del sistema (Attivo o spento)
- Si gestisce la modifica del display
- Si gestisce le varie stampe della console seriale

3.2 Considerazioni finali

Questo progetto ha rappresentato un'interessante e complessa sfida nella programmazione "bare-metal". L'approccio a basso livello mi ha permesso di padroneggiare il controllo diretto dell'hardware, gestendo manualmente i registri dei timer, gli interrupt e i protocolli di comunicazione dei sensori, nel pieno rispetto delle specifiche iniziali. Pur avendo incontrato delle difficoltà, il risultato è un sistema funzionante e ottimizzato. Per il futuro, sarebbe interessante evolvere il progetto. Si potrebbe implementare un **allarme di emergenza** con un buzzer per livelli di CO_2 critici, come ipotizzato nella relazione. Inoltre, si potrebbe sfruttare la **lettura dell'umidità**, già disponibile dai sensori DHT22 ma attualmente non utilizzata, per aggiungere una funzione di controllo anti-condensa, migliorando ulteriormente il comfort ambientale.